



POLITECNICO
MILANO 1863

MPI

Alessandro Margara

alessandro.margara@polimi.it

<https://margara.faculty.polimi.it>

Rules

- Rename the `local_minima_XX.c` file by replacing `XX` with the number of your group
- Write in the comment on top of the file
 - Your group number
 - The name of all group members
- Submit only a single `c` file with your solution
 - Submitted from the contact email provided in the group registration document

Requirements

- You are to implement a parallel MPI program that finds all local minima in a 2-dimensional integer matrix A of size $N \times N$
 - $N=256$
- N should be multiple of the number of processes used
 - Use `mpirun -np X`
 - with $X = 2$ or 4 or 8 or 16 or ...

Definitions and assumptions

- The matrix is stored in a linear array in row-major format, as shown in the figure

[0,1]	[0,2]	[0,3]	...	[0,N-1]
[1,1]	[1,2]	[1,3]	...	[1,N-1]
...	...	...	...	...
[N-1,1]	[N-1,2]	[N-1,3]	...	[N-1,N-1]



[0,1]	[0,2]	[0,3]	...	[0,N-1]	[1,1]	[1,2]	[1,3]	...	[1,N-1]	...	[N-1,N-1]
-------	-------	-------	-----	---------	-------	-------	-------	-----	---------	-----	-----------

Definitions and assumptions

- You can use function `val(A, r, c)` to access the value of matrix A in row r and column c

[0,1]	[0,2]	[0,3]	...	[0,N-1]
[1,1]	[1,2]	[1,3]	...	[1,N-1]
...	...	...	...	...
[N-1,1]	[N-1,2]	[N-1,3]	...	[N-1,N-1]



[0,1]	[0,2]	[0,3]	...	[0,N-1]	[1,1]	[1,2]	[1,3]	...	[1,N-1]	...	[N-1,N-1]
-------	-------	-------	-----	---------	-------	-------	-------	-----	---------	-----	-----------

Definitions and assumptions

- A local minimum is a point such that all adjacent points store higher values
- The green point (5) is a local minimum since the values stored in the points on top (24), bottom (64), left (33), right (32) are all higher than 5

...	...	24	...	...
...	33	5	32	...
...	...	64	...	...

Definitions and assumptions

- As a simplification, assume that only internal points can be local minima
- Global borders (in red) are not local minima by definition

[0,1]	[0,2]	[0,3]	[0,4]	...	[0,N-1]
[1,1]	[1,2]	[1,3]	[1,4]	...	[1,N-1]
...	...	...	...	...	...
...	...	...	...	...	...
[N-1,1]	[N-1,2]	[N-1,3]	[N-1,4]	...	[N-1,N-1]

Requirements

- Process 0 allocates and initializes the entire matrix A
 - Use function `generate_matrix` for that
- The detection must be performed in parallel
- At the end, process 0 collects and prints the number of local minima in each row, in order, with this format
 - Row: 0, local minima: 0
 - Row: 1, local minima: x
 - Row: 2, local minima: y
 - Row: $n-1$, local minima: 0
- Your solution should try to minimize communication and synchronization